arXiv:2401.01836v1 [cs.AI] 3 Jan 2024

NODEC: Neural ODE For Optimal Control of Unknown Dynamical Systems

Cheng Chi
University of Toronto
cheng.chi@mail.utoronto.ca

Abstract

Controlling complex dynamical systems is generally associated with minimizing certain control objectives with known dynamics under the variational calculus framework. For systems with unknown dynamics, an additional step of dynamics modeling is required. However, any inaccuracy in dynamics modeling will lead to sub-optimality in the resulting control function. Another set of approaches for controlling unknown dynamical systems - reinforcement learning, folds the dynamics modeling into controller training via value function approximation or policy gradient through extensively interacting with the environment, but it suffers from low data efficiency. To address these, we introduce NODEC, a novel framework for controlling unknown dynamical systems, which combines dynamics modelling and controller training using a coupled neural ODE model. Through an intriguing interplay between the two coupled neural networks, NODEC learns system dynamics as well as optimal controls that guides the unknown dynamical system towards target states. Our experiments demonstrate the effectiveness and data efficiency of NODEC for learning optimal control of unknown dynamical systems.

***K*eywords** Neural ODE, Dynamical System, Optimal Control

1 Introduction

Optimal control is a robust mathematical framework that enables finding control and state trajectories for dynamical systems over a period of time to minimize a performance metric [1]. The primary objective in optimal control is to determine a control law that minimizes or maximizes a performance metric, such as energy consumption, time to reach a target, or cost of operation. Solving optimal control problems can be difficult, especially for systems with nonlinear dynamics. Various methods have been developed to solve optimal control problems, including principle-driven approaches such as Pontryagin's maximum principle [2] [3], and data-driven approach based on modern machine learning and optimization techniques [4] [5] [6].

Our approach discussed in this paper falls into the data-driven category. In particular, we introduce NODEC, a novel control framework based on coupled neural ordinary differential equations that automatically learns optimal control and system dynamics for unknown dynamical systems.

1.1 Principle driven optimal control

In this section, we briefly explain the principle-driven approach for solving optimal control problems and particularly the Maximum principle [2].

The first implicit step of controlling an unknown system would be to try to formulate system dynamics as a system of ODEs, which relies on prior knowledge or requires fitting the parameters in the ODEs. Once the dynamics are estimated and represented as $f(x, u, t)$, where x represents the state variable, u is the control input, and t denotes time, we can express the system as follows:

$$\dot{x}(t) = f(x(t), u(x, t), t) \tag{1}$$

Then we formulate our control objective as minimization of the functions shown in equation (2):

$$J(u) = \int_0^T L(x(t), u(x,t), t)\, dt + M(x(T)) \tag{2}$$

where $M(x(T))$ specifies the terminal cost that penalizes the state attained at T, and $L(x(t), u(x,t), t)$ represents the instantaneous cost. The subsequent steps involve converting the constrained optimization problem (equation (1) as constraints) into unconstrained optimization via Lagrange multiplier, redefining terms inside that functional with the quantity known as Hamiltonian, and applying a first-order necessary optimality condition on the augmented objective functional. Eventually we reach the necessary condition for the optimality of the control: if (u^*, x^*) is optimal, then:

$$\begin{aligned} \dot{x}_i = \frac{\partial H}{\partial \lambda_i} \quad \dot{\lambda}_i = -\frac{\partial H}{\partial x_i} \quad \forall i \\ \lambda(T) = \frac{\partial M}{\partial x}(x(T)) \\ H(x^*(t), u^*(x,t), \lambda^*(t)) \leq H(x^*(t), u, \lambda^*(t)) \quad \forall u \end{aligned} \tag{3}$$

where $\lambda^*(t)$ are the costate variables associated with $x^*(t)$. The aforementioned optimality condition typically leads to a system of differentiable equations, and the solutions to this system contain the desired optimal control. This principle-driven approach is often analytically and computationally intractable when applied to complex dynamical systems. With the recent development in differentiable programming and physics-informed neural network [7], the maximum principle based approaches can be extended beyond analytical solutions and include higher-order terms in the optimality condition [8].

1.2 Data driven optimal control

Another class of optimal control methods relies more on differentiable optimization and less on theoretical optimality conditions. For instance, reinforcement learning is an algorithmic framework for solving optimal control problems - generally in an unknown environment. Even though certain RL algorithms can be used to approximate the solution to the Hamilton-Jacobi-Bellman (HJB) equation (necessary and sufficient condition for optimality), recent advancements in RL such as the policy gradient-based methods [9] [10] or the actor-critic based methods [11] are in a purely data-driven regime. They directly parameterize the control policy and collect a large number of interactions with the environment to build differentiable loss for improving the parameterized policy, thus skipping the dynamics modelling and the derivation of optimality condition as in the principle-driven approach. A subroutine in RL that involves dynamics modelling is known as the model-based RL, where the approximation of unknown dynamics amounts to a supervised learning problem, and the learned dynamics act as a subroutine in the original RL policy optimization process in the form of forward planning [12]. However, powerful policy optimizers exploit inaccuracies in learned dynamical models leading to adversarial examples and sub-optimal policies [13]. RL algorithms are known for having a low data efficiency [14], and the available data can be extremely limited for some dynamical systems [15].

A closely related work is done by [6], where they embed a parameterized controller inside the real dynamical functions f, and show that it is possible to generate control signals that resemble those of optimal control by minimizing a loss without relying on solving maximum principle or HJB equations. However, their work is strictly limited to controlling dynamical systems with known dynamical functions.

2 Method

In this section, we introduce our method NODEC: Neural ODE Controller, a coupled neural ODE structure that simultaneously learns optimal controls as well as dynamics of an unknown dynamical system.

2.1 Neural ODE

Neural ODE (Ordinary Differential Equation) [16] is a powerful modelling framework that leverages differential equations and neural networks to learn continuous-time dynamics. The key equation of Neural ODE is:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t) \tag{4}$$

where $h(t)$ represents the hidden state at time t, f is a neural network parameterized by θ, and t denotes the continuous time variable. The initial condition for Neural ODE is given by $h(t_0) = h_0$. The solution of the Neural ODE or the model predictions can be obtained by numerically integrating the above equation over the desired time interval. The efficient back-propagation for training neural ODE is achieved by the adjoint method. It requires solving a

backward-in-time second ODE, known as the adjoint ODE, which is associated with the original ODE. By solving the adjoint ODE, we obtain gradients of the loss function with respect to the state variables of the original ODE at each time step. These gradients are then utilized, along with the chain rule, to compute gradients with respect to the parameters of the neural network [17].

2.2 Architecture of NODEC

NODEC consists of a coupled neural ODE structure. We still parameterize the control function $u(x, t)$ through a neural network $h_\theta(x, t)$, denoted as the controller. This controller is embedded as part of the dynamics represented by another neural network $g_\gamma(x, u, t)$ denoted as the dynamics learner, and the control signal u is given by the controller. This is represented by the following equations:

$$\dot{x} = g_\gamma(x, h_\theta(x, t), t) \tag{5}$$

This is essentially a coupled neural ODE where the neural part consists of two neural networks: controller h_θ and dynamics learner g_γ. The integration in Equation (5) is conducted using a numerical solver over the dynamics specified by the coupled h_θ and g_γ, and this is how NODEC makes a prediction about the state x_T as a result of continuously controlling the state evolution using the controller in the environment with unknown dynamics. For instance, starting at x_0, NODEC predicts where the system will end up by:

$$x_T = x_0 + \int_0^T g_\gamma(x_t, h_\theta(x_t, t), t)dt \tag{6}$$

2.3 Training Loss

There are two objectives when we train our NODEC model given by $g_\gamma(x, h_\theta(x, t), t)$. The first objective is for the dynamics learner $\hat{g}_\gamma$ to approximate the real unknown dynamics $f(x, u, t)$: the integration in Equation (5) gives us the model prediction of the trajectory roll-out, and we want this prediction to match the real trajectory coming from the environment (the real dynamics) using the same controller h_θ. The second objective is for the controller h_θ: we want to satisfy the optimal control objectives, for example, by continuously applying control signal u given by h_θ, the end state in the predicted trajectory x_T using the dynamics learner g_γ (which is also the end state in the real trajectory if g_γ is well trained) should be close to our target state x^*.

These two objectives manifest the training loss, which is presented in Figure 1 below:

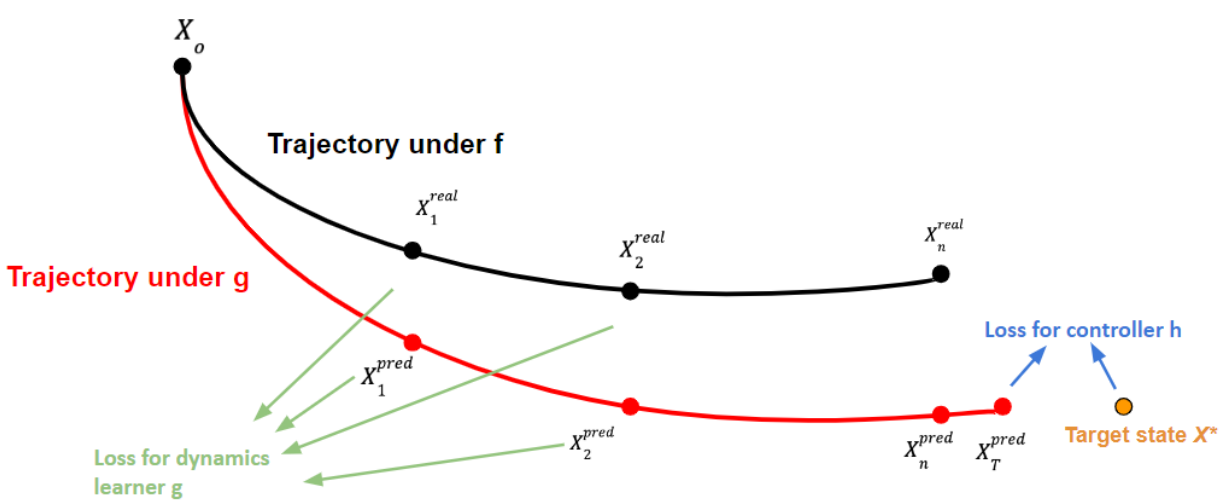

Figure 1: Training loss for dynamics learner g_γ and controller h_θ

Starting from the same initial state x_0, the predicted trajectory under $g_\gamma(x, h_\theta(x, t), t)$ is shown in red, and the real trajectory coming from applying the same controller h_θ in the environment or in real dynamics f is shown in black. As g_γ should approximate f, the mean squared error (MSE) between these two trajectories (shown in green) is used to update g_γ to reflect the dynamic learning objective. In contrast, the MSE loss between x_T^{pred} and x^* (shown in blue) is used to train controller h_θ to reflect the optimal control objective. Note that the controller loss can also be constructed

using the whole trajectory, which is the case for our CartPole experiment in Section 3, and we use a single end state here for illustrative purpose.

By setting up the losses in this manner, the training of the controller is conducted exclusively through simulation using roll-out by g_γ (as depicted by the blue arrows in Figure 1), and the real trajectories data from the environment is only required for training dynamics learner. This significantly improves data efficiency, which is demonstrated by our results in section 3 where we successfully learn to control the CartPole system with less than 60 trajectories collected from the environment. This is extremely data-efficient compared to traditional RL methods that use hundreds or thousands of trajectories or episodes for CartPole [18].

The relationship between controller h_θ and dynamics learner g_γ is intriguing. On one hand, the dynamics learner determines whether the controller training is effective or not, as it is purely using trajectory roll-outs under g_γ. If g_γ has already been well-trained and sufficiently approximates f, the controller training process genuinely enhances the controller's performance in terms of our control objective: guiding states towards the target state under the real and unknown dynamics f. If the dynamics learner is poorly trained, the controller may exploit the learned dynamics and produce ineffective controls when deployed in the real dynamics.

On the other hand, the controller plays a vital role in shaping the trajectory's roll-out within the state space, as its outputs serve as inputs to both the dynamics learner $g\gamma(x, h\theta(x,t), t)$ and the true dynamics $f(x, h_\theta(x,t))$. As indicated in Figure 1, these trajectories roll-outs establish the training loss for the dynamics learner. Therefore, the controller essentially determines which portion of the state space dynamics the dynamics learner would fit. Through this mechanism, the controller effectively directs the dynamics learner's training process.

2.4 Alternative Training

A naive approach for training NODEC model $g_\gamma(x, h_\theta(x,t), t)$ is a two-stage training process. In the first stage, the dynamics learner g_γ is trained to accurately capture the underlying dynamics f by leveraging a diverse set of controllers (ideally all possible control functions) for generation the input controls u. In the second stage, the controller h_θ is trained within the learned dynamics model g_γ to attain optimal control, which is similar to the approach described in [6] for learning optimal control with known dynamics.

There are two shortcomings with this two-stage training approach: First, once the training of the controller begins, no further updates will be made to the learned dynamics g_γ. Consequently, the controller has the potential to exploit the fixed learned dynamics and generate adversarial controls if g_γ is poorly fit. This raises uncertainty regarding whether the set of controllers used in the initial stage of training g_γ is truly diverse enough for exploring all the state space of f for a good fit of g_γ.

Furthermore, it is unnecessary to precisely fit f over all state space. As mentioned earlier, the controller dictates which portion of the vector field the dynamics learner should approximate. Since our goal is to train the controller to be optimal, our focus is on thoroughly exploring the vector field of f that is relevant to the optimal controller and controllers from previous training steps only. Therefore, we do not need to generate all possible control functions (and we may not even be able to generate near-optimal controllers from the start) for training the dynamics learner in the first stage. Instead, the dynamics learner only needs to approximate the vector field locally, specifically in the regions where the current controller is leading, which is shown in Figure 6 Appendix A.

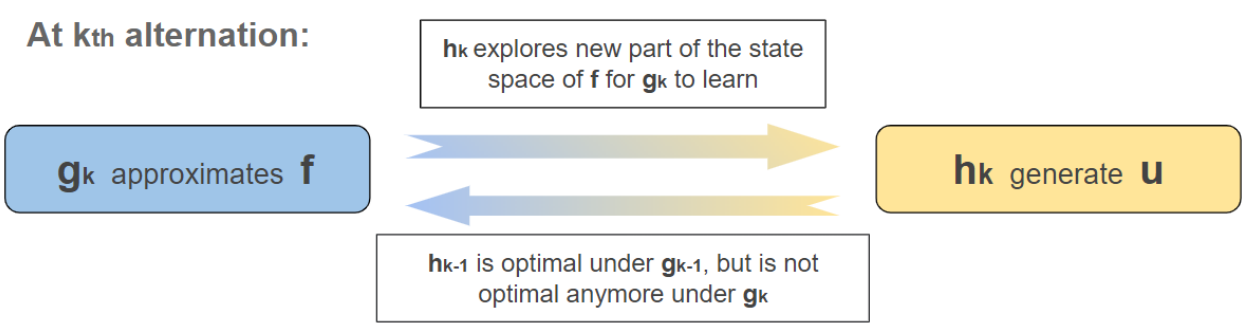

Figure 2: The k_{th} alternation of the dynamics learner g_γ and controller h_θ alternative training

Algorithm 1 Alternative Training of NODEC Model

Input: Initial state x_0, Target state x^*, Initialized neural networks g_γ^0 and h_θ^0, Number of alternations K between training g_γ and h_θ, Training iterations n_g and n_h within each alternation for training g_γ and h_θ respectively, Interactive environment f
Initialize $k \leftarrow 0$
while $k < K$ **do**
  Deploy h_θ^k in environment f starting from x_0
  Collect real trajectories T_{real}
  Train Dynamics Learner
  For n_g update steps:
    Deploy h_θ^k in g_γ^k starting from X_0
    Collect predicted trajectories T_{pred}
    Update g_γ^k with dynamics fitting loss:
      $L_g = MSE(T_{pred}, T_{real})$
  Train Controller
  For n_c update steps:
    Deploy h_θ^k in already updated g_γ^k starting from X_0
    Collect predicted trajectories T_{pred}
    Update g_γ^k with control loss:
      $L_c = MSE(T_{pred}, x^*)$
  $k \leftarrow k + 1$
end while

This naturally leads to the alternative training scheme depicted in Figure (2) for training NODEC model $g_\gamma(x, h_\theta(x, t), t)$. In this scheme, we alternate the training of the two neural networks, g_γ and h_θ, for in total of K iterations. We denote the neural networks at the k_{th} alternation as g_γ^k and h_θ^k, respectively.

Within each alternation, such as the k_{th} alternation, we first train the dynamics learner g_γ^k using numerous gradient steps to obtain g_γ^{k+1} while keeping the controller g_γ^k fixed. After this training, this new dynamics learner has learned to approximate the real dynamics near the trajectories that the current controller h_θ^k leads to.

Subsequently, we fix the dynamics learner and focus on training the controller h_θ^k (only optimal under g_γ^k) to be optimal under g_γ^{k+1} in terms of the control objective, resulting in a new controller h_θ^{k+1}. However, this new controller would lead to a different portion of the state space whose dynamics may not be accurately approximated by the current dynamics learner g_γ^{k+1}, as g_γ^{k+1} only approximates the true dynamics well near the trajectories associated with the previous controller $h\theta^k$. Therefore, we proceed to the next iteration. This alternative process continues until convergence is achieved for both the dynamics learner and the controller.

The algorithm for alternative training with the corresponding loss definitions is outlined in Algorithm 1. It is important to note that, for the purpose of illustration, the algorithm is presented with only one initial state. However, it can be readily extended to handle a batch of initial states.

3 Experiments

3.1 Control task: Linear System

We now investigate the control performance of NODEC applied to a linear dynamical system described by $f(x, u) = Ax + Bu$.

Dynamical system and control objectives

For such systems, analytical optimal control (OC) inputs can be derived using the Maximum Principle described in Equation (3). The optimal control input $u^*(t)$ is given by the equation:

$$u^*(t) = B^\top e^{A^\top (T-t)} W(T)^{-1} v(T) \tag{7}$$

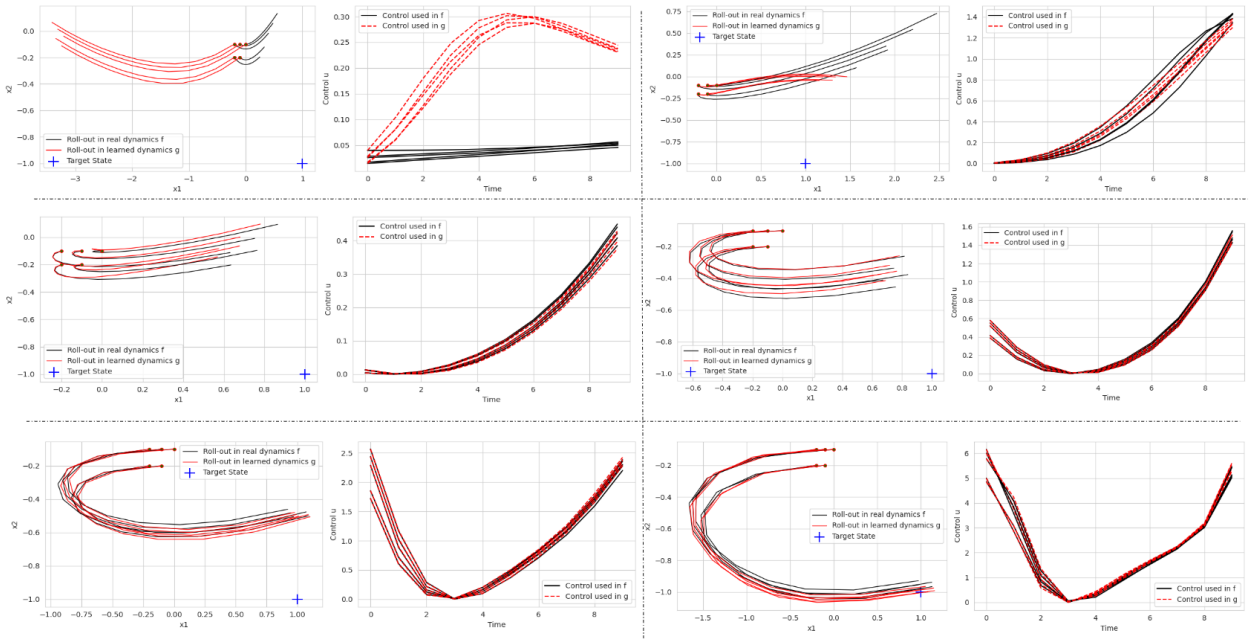

Figure 3: The full training process of NODEC for Ax+Bu task

.

$v(T) = x(T) - e^{A(T-0)}x(0)$ represents the difference between the target state $x(T)$ and the initial state $x(0)$ under free evolution, as denoted in Equation (7). The matrix $W(T)$ corresponds to the controllability Gramian, defined as:

$$W(T) = \int_0^T e^{At} BB^\top e^{A^\top t} dt. \tag{8}$$

This control task is the same task [6] used, but we assume the dynamics is unknown, and our NODEC model $g_\gamma(x, h_\theta(x,t), t)$ does not have access to the true dynamics $f(x, u)$. We aim to evaluate whether our NODEC can learn the analytical optimal control without the knowledge of the true dynamics. We set problem dynamics parameters: A = $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and the driver matrix B = $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$. We use a batch of uniformly random five initial states, the target state is set to be $x^* = (1, -1)$, and control horizon is set to be $T = 1$. Therefore, our control objective is to bring the system state x_T at $T = 1$ closer to the target state starting from those initial states x_0 in the randomly generated batch.

Algorithm set-up

For solving this linear control task with our NODEC model $g_\gamma(x, h_\theta(x,t), t)$, we employ a controller h_θ and a dynamics learner g_γ with one hidden layer consisting of 30 neurons each. The output layer of both neural networks uses the identity activation function, while the hidden layer uses the tanh non-linear activation function.

To train our model using Algorithm 1, we set the training alternation parameter to K = 10, indicating that it only needs to collect at most ten sets of real trajectories from the environment or the true dynamics. Within each alternation, we train both the dynamics learner and the controller for 10000 gradient steps ($n_g, n_c = 10000$), using a constant learning rate of 0.005. The training process leverages the built-in auto-differentiation functions in Jax.

Results

Figure (3) demonstrate the full training process of NODEC when controlling the $Ax + Bu$ linear system. It provides an overview of the full alternative training process, consisting of six pairs of plots separated by dashed lines. Each pair corresponds to the results obtained in a specific training alternation k. The top-left pair corresponds to the initial training alternation ($k = 0$), while the bottom-right pair represents the fully trained model after $K = 10$ alternations between the dynamics learner and the controller training. The pairs in between correspond to intermediate training alternations.

In each pair of plots, the left plot displays the trajectory roll-outs under the real dynamics f (shown in black) and the current dynamics learner g_γ^k (shown in red), using the controller h_θ^k for the current alternation k. Meanwhile, the right

plot shows the control signal generated along the trajectories using the current controller h_θ^k. These control signals corresponds to the outputs of $h_\theta^k(x,t)$, where x represents the states in the trajectories under f and g_γ^k, respectively.

The final trajectory plot at the bottom right corner reveals that the trained NODEC controller h_θ^K successfully guides the system from various initial states to the target state $x^* = (1, -1)$, in both learned dynamics and real dynamics (which shows that the training of the g_γ^K is successful as well). The training losses for dynamics learner and controller corresponds to these six alternations (each alternation we take n_c, n_g gradient steps) are shown in Figure (8) Appendix C.

Moreover, it is fascinating to observe that the controller and dynamics learner converge almost simultaneously: as the controller performs better (end states in trajectories are closer to the target state), the dynamics learner adjusts to fit the dynamics more accurately (deviation between black and red trajectories are smaller). As discussed in Section 2.4, an intriguing interaction unfolds between the controller and the dynamics learner. The controller effectively guides the dynamics learner's learning process by leading it to different regions of the vector field in f that require accurate fitting. Conversely, the dynamics learner imposes constraints on the controller by specifying the dynamics that the controller will operate within. In essence, they mutually shape each other's training objectives and supervise one another during each alternation k. As the alternations progress, the quality of this mutual supervision between the two neural networks continually improves. As the controller's performance enhances, the dynamics learner learns to approximate dynamics that are closer to optimal trajectories in the state space. Simultaneously, as the dynamics learner becomes more proficient, the controller's training becomes more effective in terms of the objective of steering the states closer to the target states, as the dynamics that the controller operates in is closer to the true dynamics. This iterative feedback loop reinforces their collaborative training and leads to increasingly refined coordination between the controller and the dynamics learner.

These findings once again highlight the captivating interactions and relationships between the controller h_θ and dynamics learner g_γ in our NODEC model $g_\gamma(x, h_\theta(x,t), t)$. Through this enthralling interplay between the two neural networks, we can solve optimal control tasks without any prior knowledge of the system dynamics.

3.2 Control Task: CartPole

We now investigate the control performance of NODEC applied to a non-linear dynamical system known as CartPole or inverted pendulum.

Dynamical system and control objectives

The CartPole system is a classic problem in control theory and reinforcement learning. It consists of a pole that is attached to a cart, which can move along a frictionless track.

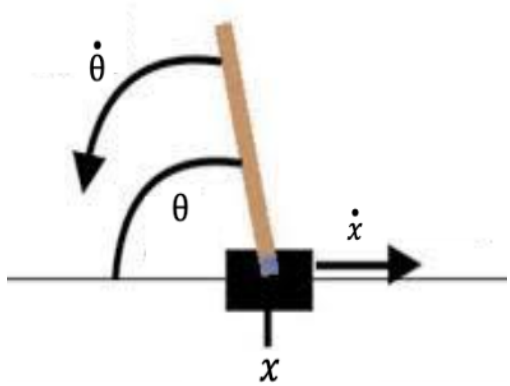

Figure 4: CartPole dynamical system

The state of the system is typically represented by four variables: $(x, \dot{x}, \theta, \dot{\theta})$ that represents the position of the pole, the horizontal velocity of the pole, the pole angle with respect to the vertical axis, and the pole's angular velocity, respectively.

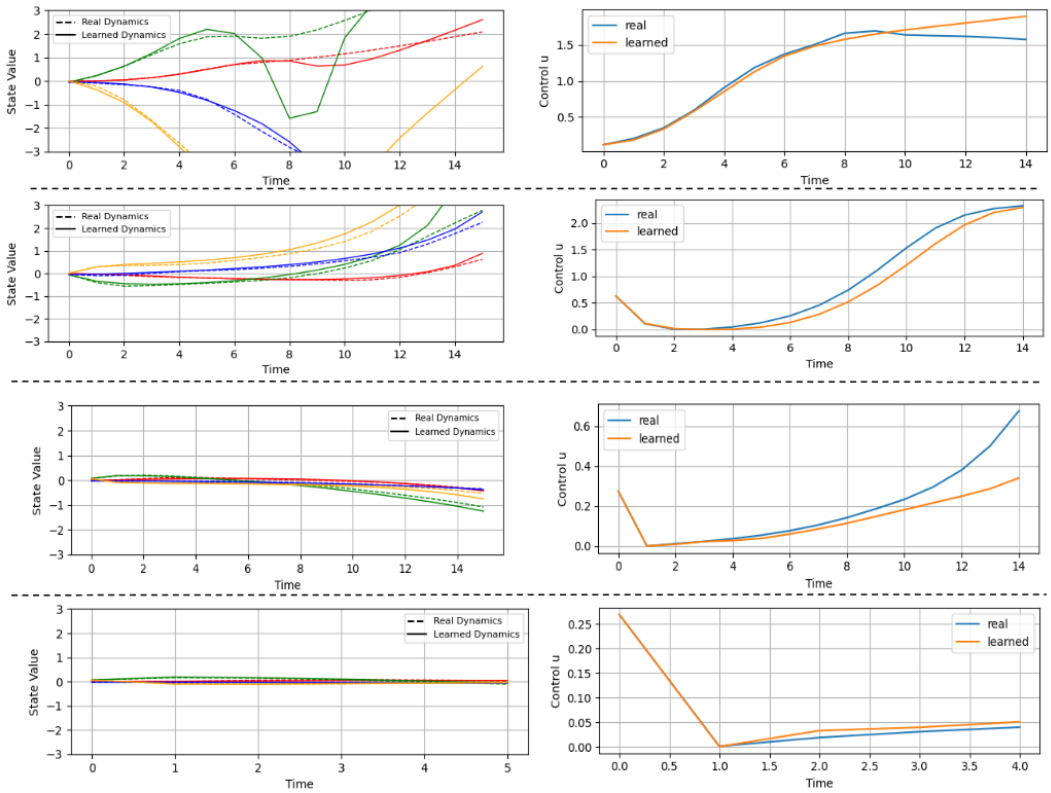

Figure 5: The full training process of NODEC for CartPole task

.

The control objective is to maintain the pole in an upright position (keep system state at (0,0,0,0)) by applying appropriate control inputs, which is the horizontal force applied to the cart. The CartPole system is a challenging problem because it is inherently unstable. Small changes in the control inputs or disturbances can cause the pole to fall.

The dynamics of the system $f(x, \dot{x}, \theta, \dot{\theta})$ that inputs the four-dimensional state and returns $\dot{x}, \ddot{x}, \dot{\theta}, \ddot{\theta}$ (the instantaneous change to the state) is given by the following system of differential equations [19]:

$$
\begin{aligned}
\dot{x} &= \dot{x} \\
\ddot{x} &= \frac{F + m_p\, l\, (\dot{\theta}^2 \sin\theta - \ddot{\theta}\cos\theta)}{m_c + m_p} \\
\dot{\theta} &= \dot{\theta} \\
\ddot{\theta} &= \frac{g\sin\theta + \cos\theta\left(\frac{-F - m_p\, l\, \dot{\theta}^2 \sin\theta}{m_c + m_p}\right)}{l \cdot \left(\frac{4}{3} - \frac{m_p \cos^2\theta}{m_c + m_p}\right)}
\end{aligned}
\tag{9}
$$

Here, F represents the control input applied to the cart, which is determined by the controller. The gravitational acceleration is $g = 9.8$. The masses of the cart and the pole are represented by $m_c = 1.0$ and $m_p = 0.1$ respectively. The length of the pole is $l = 0.5$ (half of the actual pole length). Equation (9) shows that the system dynamics are highly non-linear. Note that the first line $\dot{x} = \dot{x}$ simply means that the first dimension of the output of f (how x should change) is directly equal to the third dimension of the input to f.

An important observation is that the control loss in our NODEC model for the CartPole task has transitioned from a single-end-state loss to an integral loss. The control loss now measures the mean squared error between the desired state (0,0,0,0) and all states along the trajectories. This is because the objective is to maintain the pole in an upright position throughout the entire trajectory. This can be easily incorporated into our NODEC alternative training algorithm by changing line 17 of Algorithm 1.

Algorithm Set-up

For solving the CartPole control task with our NODEC model $g_\gamma(x, h_\theta(x,t), t)$, we set up our controller h_θ with a single hidden layer consists of 64 neurons and one-dimensional output for control F, and our dynamics learner g_γ with two hidden layers consisting of 64 and 32 neurons respectively and having a four-dimensional output corresponds to $\dot{x}, \ddot{x}, \dot{\theta}, \ddot{\theta}$. The output layer for both h_θ and g_γ uses the identity activation function, while the hidden layer uses the tanh non-linear activation function. We set the training alternation parameter to $K = 12$ in Algorithm 1. During each alternation, we train the controller for $n_c = 2000$ gradient steps with a learning rate of 0.0001. The training of the dynamic learner follows a multi-stage approach that enhances the stability of training. Firstly, we minimize the mean squared errors of trajectory roll-outs from $t = 0$ to $t = 0.5$ for 2000 gradient steps, using a learning rate of 0.005. Next, we extend the training to cover the time range from $t = 0$ to $t = 1$ for an additional 2000 gradient steps, with a learning rate of 0.001. Finally, we continue the training from $t = 0$ to $t = T$ for another 2000 gradient steps, employing a learning rate of 0.0001. This multi-stage training can be seen in Figure (9) Appendix C, where the dynamics learner loss plots have spikes for every 2000 gradient steps. On the other hand, the dynamic trainer follows a multi-stage training approach for the purpose of stability. AT first, the time step is set to $t = 0.5$ with $n_c = 10000$ gradient steps and a learning rate of 0.001. Then, the time step is extended to a time range of $t = 0.5$ to $t = 1$ with an additional 10000 gradient steps n_c and a learning rate of 0.0001.

Results

Figure (5) indicates the full training process of NODEC for controlling the CartPole system with four pairs of plots. The pair on the top corresponds to the initial training alternation ($k = 0$), while the bottom represents the fully trained model after $K = 12$ alternations. The pairs in the middle correspond to intermediate training alternations. In each plot pair, the left plot illustrates the trajectory roll-outs under the real dynamics f (shown in dashed lines) and under the current dynamics learner g_γ^k using the controller. Since the states in this case have four dimensions, we represent the trajectory roll-outs by plotting the values of each state dimension over time, rather than directly visualizing the trajectories in the state space. It is important to note that we visualize the results for a single initial state from the batch of data. The right plot in each pair displays the control signals generated along the trajectories.

We can observe that NODEC controller h_θ^K successfully controls the CartPole system after K alternations, as it keeps the system evolution near the target state (0,0,0,0) which corresponds to the upright position of the pole, and the trajectory roll-outs in learned dynamics are close to real dynamics. Again, the training losses for dynamics learner and controller correspond to these four alternations are shown in Figure (9) Appendix C.

One thing to note here is the data efficiency of NODEC algorithm. As indicated in Algorithm 1 line 5, we only need to interact with the real environment and collect real trajectories at each alternation k for each initial state in our batch. Therefore, we only need to collect in total $K * N$ trajectories or episodes from the real environment where N is the batch size. For this particular experiment, we only collected less than 60 trajectories before successfully solving the CartPole tasks, which is more data-efficient compared to RL methods that uses hundreds or thousands of trajectories.

4 Conclusion

We introduced a novel framework denoted as NODEC that consists of a coupled neural ODE for controlling dynamical systems with unknown dynamics. This is achieved via an alternative training between two neural networks in our NODEC model: the dynamics learner and the controller. Through an intriguing interplay and mutual supervision between these two neural networks, the model learns the optimal control as well as the system dynamics. We applied NODEC in different dynamical systems and demonstrated the effectiveness of NODEC for learning the optimal control with high data efficiency.

This manuscript presents a detailed and technical exposition of the method introduced in Chapter 3 ('TEL for Optimal Control: NODEC') of the thesis titled 'Theory Embedded Learning' [20]. For further reference, the relevant code implementations can be accessed here

References

[1] Wiley Online Library. Optimal control applications and methods. *Wiley Online Library*, 2023.

[2] Pontryagin L Boltyanskii V. Gamkrelidze R. Mishchenko. *Mathematical Theory of Optimal Processes*. 1961.

[3] E. McShane. The calculus of variations from the beginning through optimal control theory. *SIAM J. Control Optim*, 26:916–939, 1989.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[5] Richard S Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. *IEEE Transactions on Neural Networks*, 4(5):782–791, 1993.

[6] Lucas Böttcher, Nino Antulov-Fantulin, and Thomas Asikis. Ai pontryagin or how artificial neural networks learn to control dynamical systems. *Nature Communications*, 13(1):333, 2022.

[7] G. E. Karniadakis and et al. Physics-informed machine learning. *Nat. Rev. Phys.*, 3:422–440, 2021.

[8] W. Jin, Z. Wang, Z. Yang, and S. Mou. Pontryagin differentiable programming: an end-to-end learning and control framework. In *33rd International Conference on Advances in Neural Information Processing Systems, NeurIPS*, 2020.

[9] John Schulman, Sergey Levine, Philipp Moritz, Michael I Jordan, and Pieter Abbeel. Trust region policy optimization. *arXiv preprint arXiv:1502.05477*, 2015.

[10] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12(1):1057–1063, 2000.

[11] Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12(1):1008–1014, 2000.

[12] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. 2020.

[13] Michael Janner, Yilun Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, May 2022. DOI: 10.48550/arXiv.2205.09991, Corpus ID: 248965046.

[14] Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations (ICLR)*, 2021.

[15] Amine Mohamed Aboussalah, Minjae Kwon, Raj G Patel, Cheng Chi, and Chi-Guhn Lee. Recursive time series data augmentation. In *International Conference on Learning Representations (ICLR)*, February 2023. Published: 01 Feb 2023, Last Modified: 13 Feb 2023.

[16] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, pages 6571–6583, 2018.

[17] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31:6571–6583, 2018.

[18] Swagat Kumar. Balancing a cartpole system with reinforcement learning, 2020.

[19] Razvan V. Florian. Correct equations for the dynamics of the cart-pole system. 2005.

[20] Cheng Chi. *Theory Embedded Learning*. PhD thesis, Mechanical and Industrial Engineering, University of Toronto, Nov 2023.

A Alternative training motivation

Figure 6 below depicts the trajectory roll-outs under $g\gamma(x, h\theta(x,t), t)$ using already well-trained $g\gamma$ and $h\theta$ (after K alternations in Figure 3. The figure also includes visualizations of the true vector field of f and the vector field of the learned dynamics g. While the vector field approximation may not be ideal across most of the state space, it fits perfectly in the vicinity of the trajectory roll-outs (as indicated by the black circles where dark red and dark blue vector fields are aligned). This demonstrate the points we made in section 2.4: we do not need a perfect fit dynamics learner, but rather just a good local approximation of the dynamics near the state space where the current controllers can lead to. And this local good approximation already suffices for learning the optimal controls.

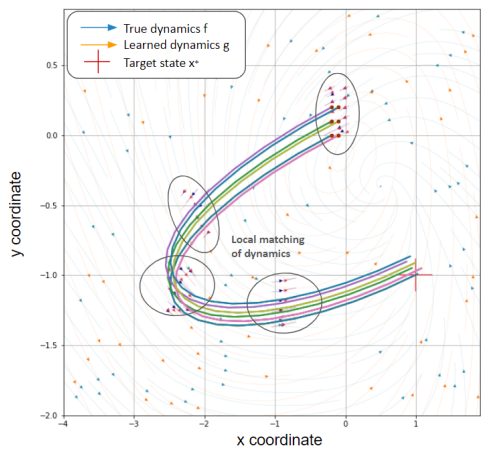

Figure 6: Vector field visualization of trained NODEC for Ax + Bu task

B Compare with analytical optimal control

Here we compare the trained NODEC controller in section 3 Figure 3 with the analytical optimal controller given by Equation 7. The left plot in Figure 7 above shows the trajectory roll-out in learned dynamics (which is almost the same as the real dynamics as indicated by the last pair of plots in Figure 3). The black trajectories are roll-out using trained controller with NODEC (the same trajectories in the last pair in Figure 3), while the blue trajectories are roll-out using optimal controllers. The high similarity between trajectory roll-outs as well as control signals indicate that through alternative training, our NODEC model can indeed learn optimal controls without any prior knowledge about system dynamics.

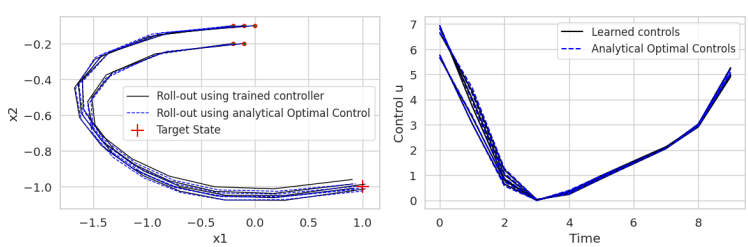

Figure 7: Comparison of NODEC controller with optimal controller

C Dynamic learner and Controller training loss

Please note that Figure (8) contains a total of 6 plot pairs, corresponding to the same alternations depicted in Figure (3) with the exact layout.

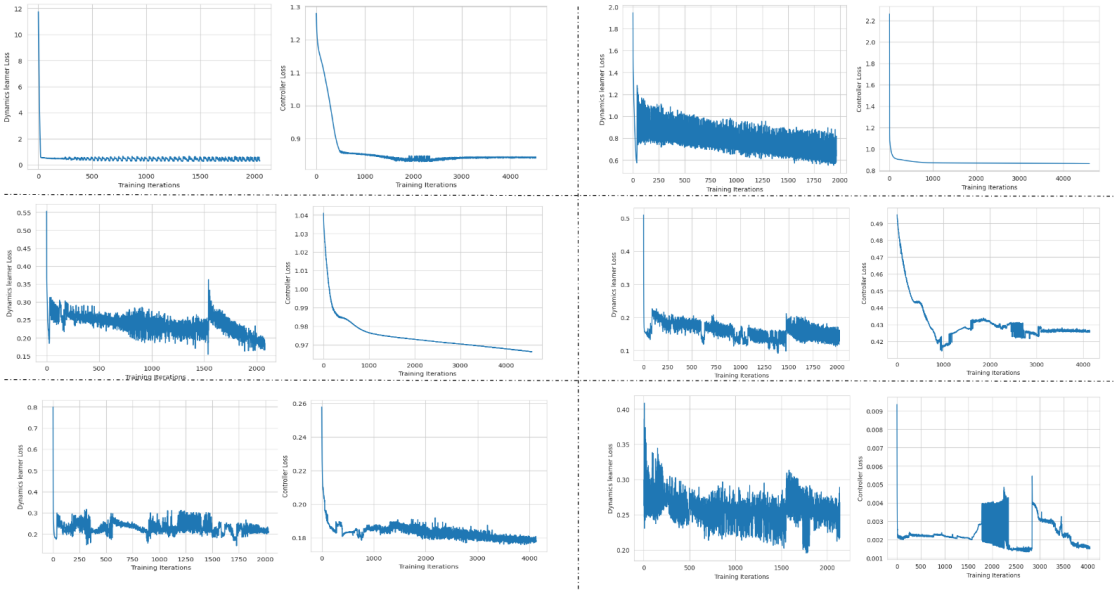

Figure 8: Dynamics learner training loss and controller training loss over K alternations for **Ax + Bu** task

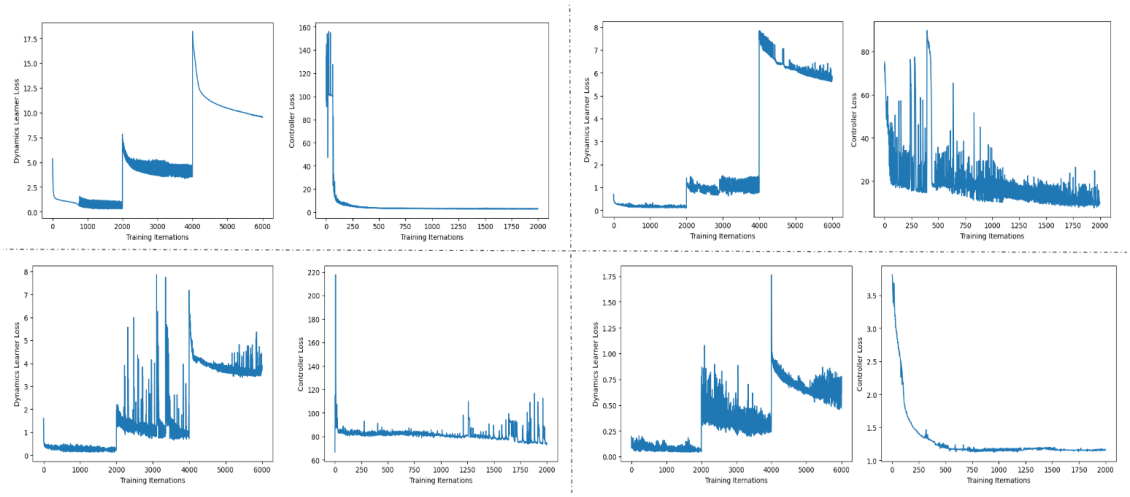

Figure 9: Dynamics learner training loss and controller training loss over K alternations for **CartPole** task

Please note that Figure (9) contains a total of 4 plot pairs, corresponding to the same alternations depicted in Figure (5). The sequence of the plot pairs is as follows: top left, top right, bottom left, bottom right.